

Case Statement in SQL: Explanation and Examples

The **CASE** statement in SQL is a powerful conditional expression that allows you to specify different outcomes based on various conditions. It's similar to **if-then-else** statements in other programming languages and is crucial for adding logic and flexibility to your queries. The **CASE** statement can be used in **SELECT** clauses, **WHERE** clauses, **ORDER BY** clauses, and **GROUP BY** clauses.

Syntax of the CASE Statement

There are two main forms of the **CASE** statement: simple **CASE** and searched **CASE**.

Simple CASE Expression

The simple **CASE** expression compares an expression to a set of simple expressions to determine the result.

CASE expression

```
WHEN value1 THEN result1
```

```
WHEN value2 THEN result2
```

```
...
```

```
ELSE default_result
```

```
END
```

Searched CASE Expression

The searched **CASE** expression evaluates a set of Boolean expressions to determine the result.

CASE

```
WHEN condition1 THEN result1
```

```
WHEN condition2 THEN result2
```

...

ELSE default_result

END

The **ELSE** clause is optional. If omitted and no **WHEN** condition is met, the **CASE** statement returns **NULL**.

Use Cases and Examples

Here are various scenarios where the **CASE** statement proves invaluable, illustrated with practical examples.

1. Categorizing Data

You can use **CASE** to categorize data into different groups based on specific criteria.

Example: Categorizing Products by Price

Suppose you have a **Products** table with a **Price** column, and you want to categorize products as 'Expensive', 'Moderate', or 'Affordable'.

ProductID	ProductName	Price
1	Laptop	1200.00
2	Mouse	25.00
3	Keyboard	75.00
4	Monitor	300.00
5	Speakers	50.00

SELECT

ProductName,

```
Price,  
  
CASE  
  
    WHEN Price >= 200 THEN 'Expensive'  
  
    WHEN Price >= 50 AND Price < 200 THEN 'Moderate'  
  
    ELSE 'Affordable'  
  
END AS PriceCategory  
  
FROM  
  
Products;
```

Result:

ProductName	Price	PriceCategory
Laptop	1200.00	Expensive
Mouse	25.00	Affordable
Keyboard	75.00	Moderate
Monitor	300.00	Expensive
Speakers	50.00	Moderate

2. Custom Sorting

The **CASE** statement can be used within an **ORDER BY** clause to define custom sorting logic.

Example: Sorting Employees by Department and then by a custom order

Imagine you want to sort employees, prioritizing a specific department (e.g., 'Sales') at the top, then other departments alphabetically.

EmployeeID	EmployeeName	Department	HireDate
101	 Person	Sales	 Date
102	 Person	Marketing	 Date
103	 Person	Engineering	 Date
104	 Person	Sales	 Date
105	 Person	HR	 Date

```
SELECT
    EmployeeName,
    Department
FROM
    Employees
ORDER BY
    CASE WHEN Department = 'Sales' THEN 1
        WHEN Department = 'Marketing' THEN 2
        ELSE 3
    END,
    Department,
    EmployeeName;
```

Result (Example):

EmployeeName	Department
Person	Sales
Person	Sales
Person	Marketing
Person	Engineering
Person	HR

3. Conditional Aggregation

CASE statements are excellent for performing conditional aggregations, such as counting specific types of records or calculating sums based on certain criteria within a single query.

Example: Counting Orders by Status

You want to count the number of 'Completed' and 'Pending' orders from an **Orders** table.

OrderID	OrderStatus	OrderDate	Amount
1	Completed	Date	150.00
2	Pending	Date	200.00
3	Completed	Date	75.00
4	Cancelled	Date	100.00
5	Pending	Date	300.00

```
SELECT
```

```
    SUM(CASE WHEN OrderStatus = 'Completed' THEN 1 ELSE 0 END) AS
    CompletedOrders,
```

```
    SUM(CASE WHEN OrderStatus = 'Pending' THEN 1 ELSE 0 END) AS PendingOrders,
```

COUNT(*) AS TotalOrders

FROM

Orders;

Result:

CompletedOrders	PendingOrders	TotalOrders
2	2	5

4. Handling NULL Values

You can use **CASE** to provide a default value or a meaningful message when a column contains **NULL**.

Example: Displaying a default for NULL values

If a **Customer**'s **Email** is **NULL**, display 'No Email Provided'.

CustomerID	CustomerName	Email
1	Anil Kumar	Anil@ example.com
2	Donald	NULL
3	Jack	Jack@ another.com

SELECT

CustomerName,

CASE

WHEN Email IS NULL THEN 'No Email Provided'

ELSE Email

END AS ContactEmail

FROM

Customers;

Result:

CustomerName	ContactEmail
Anil Kumar	Anil@ example.com
Donald	No Email Provided
Jack	Jack@ another.com

5. Dynamic Column Names (Pivoting)

While not a direct pivot function, **CASE** can be used with aggregation functions to simulate pivoting, where rows are transformed into columns.

Example: Sales per Month

If you have sales data and want to display total sales for each month as separate columns.

SaleID	SaleDate	Amount
1	Oct 8, 2025	100.00
2	Oct 7, 2025	150.00
3	Jan 14, 2026	200.00
4	Mar 25, 2027	120.00

SELECT

SUM(CASE WHEN STRFTIME('%m', SaleDate) = '01' THEN Amount ELSE 0 END) AS
JanuarySales,

```
SUM(CASE WHEN STRFTIME('%m', SaleDate) = '02' THEN Amount ELSE 0 END) AS  
FebruarySales,
```

```
SUM(CASE WHEN STRFTIME('%m', SaleDate) = '03' THEN Amount ELSE 0 END) AS  
MarchSales
```

```
FROM
```

```
Sales;
```

Result (Example):

JanuarySales	FebruarySales	MarchSales
100.00	270.00	200.00

(Note: `STRFTIME` is a SQLite function for formatting dates. The equivalent might be `FORMAT(SaleDate, 'MM')` in SQL Server, `TO_CHAR(SaleDate, 'MM')` in Oracle, or `DATE_FORMAT(SaleDate, '%m')` in MySQL.)

6. Conditional Updates

The `CASE` statement can also be used in `UPDATE` statements to conditionally modify data.

Example: Adjusting Employee Salaries

Increase salaries by 10% for 'Sales' department employees and by 5% for 'Marketing' department employees.
`UPDATE Employees`

```
SET Salary = Salary *
```

```
CASE
```

```
    WHEN Department = 'Sales' THEN 1.10
```

```
    WHEN Department = 'Marketing' THEN 1.05
```

```
    ELSE 1.00 -- No change for other departments
```

```
END;
```


7. Data Validation and Error Handling

You can use **CASE** to identify and flag data that doesn't meet certain criteria.

Example: Identifying Incomplete Records

Mark records as 'Incomplete' if essential fields are missing. **SELECT**

 OrderID,

 CustomerID,

 OrderDate,

CASE

WHEN CustomerID **IS NULL** **OR** OrderDate **IS NULL** **THEN** 'Incomplete Record'

ELSE 'Complete Record'

END AS RecordStatus

FROM

 Orders;

Conclusion

The **CASE** statement is a versatile and essential tool in SQL for implementing conditional logic. Whether you need to categorize data, perform custom sorting, aggregate conditionally, handle **NULL** values, or even simulate pivoting, **CASE** provides the flexibility to craft powerful and dynamic queries. Mastering its use will significantly enhance your ability to manipulate and analyze data effectively.